ECE373 PROJECT
Final Report

Group 3:

Chen, ChangYuan

Prato, Kameron Dylan

Yuan, Konray

YouTube Link

https://youtu.be/SzET5pRDcLQ

2025 Dec

Table of Contents

Deliverable 1: Software 3

Deliverable 2: File Report 3

2.1. Problem Description 3

2.2. Project Development Roadmap 4

2.3. Project Development Challenges 4

2.3.1 Learning Curve 4

2.3.2 Collaboration 4

2.4 Use Cases 5

2.4.1 Individual Use-Case descriptions 5

2.4.2 Use-Case State Model 8

2.4.3 Interaction Model (Sequence Diagrams) 13

2.5 Domain Class Model 17

2.5.1 Default Package (including UI Classes) 18

2.5.2 Model Classes 19

2.6 Testing 23

2.6.1 Unit Testing 23

2.6.2 Module Testing 24

2.6.3 Use-Case Testing 25

2.7 User Interfaces (UIs) 32

2.7.1 Customer UI 32

2.7.2 Cashier UI 32

2.7.3 Cooker UI 33

2.7.4 Orders Info UI (Bonus Feature) 33

2.7.5 Store Info UI (Bonus Feature) 35

2.8 Software Install and Launch 36

Deliverable 3: YouTube Video 38

Deliverable 1: Software

The prototype code is included in the ZIP file (ECE373_JavaProject_Group3.zip).

Deliverable 2: File Report

2.1. Problem Description

Design a software for a fast-food drive-thru system which is including users, cashiers and cookers. The software is a window application (Figure 1.1). It can be installed on the system main / central computer and a station computer for users, cashiers, and cooker.

The software displays food menu to users. Users enter the selected food items for ordering. Users pay the order, and then cashiers place the order for preparing. The order will be displayed in the ordering system.

Each cashier station has a computer to accept payments, print receipts, set order status, and display the order for preparing the food.

Each cooker station has a computer to set order status based on the preparing phases. Cooker can help customer to pick up an order.

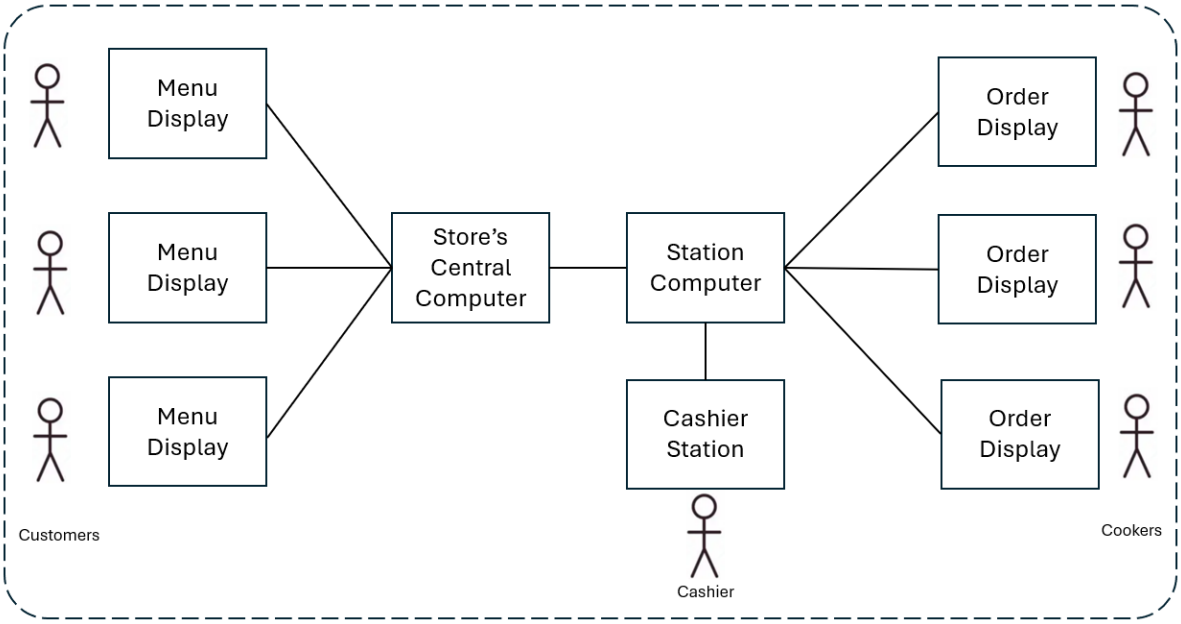


Figure 1.1 Fast-Food Drive-Thru Ordering System

2.2. Project Development Roadmap

Milestone	**Deliverables**	**Due Date**
Task 1	Detailed Problem Descriptions, Use cases, User Interface (UI) Concept	10/31
Task 2	Domain Class Model and State Model	11/4
Task 3	Milestone Review: a Basic Prototype and Initial Testing	11/25
Class Activity 5	Project Milestone Review	12/5
Task 4	Software, Final Report, YouTube Video	12/10
Show-and-tell Presentation	3 minutes presentation	12/10

2.3. Project Development Challenges

2.3.1 Learning Curve

Java Swing and Art libraries are new to all of us. We learned Java GUI development during the practices of class Lab4 and Homework4 activities.

2.3.2 Collaboration

Team collaboration needs all team members' support. We are lucky to have a team that everyone is proactive. We collaborate the document revise through emails; we collaborate coding development through GitHub.

2.4 Use Cases

Following figure is the use case diagram for the project. There are three actors (customer, cashier, and cooker). We identify a few use cases which will be explained next.

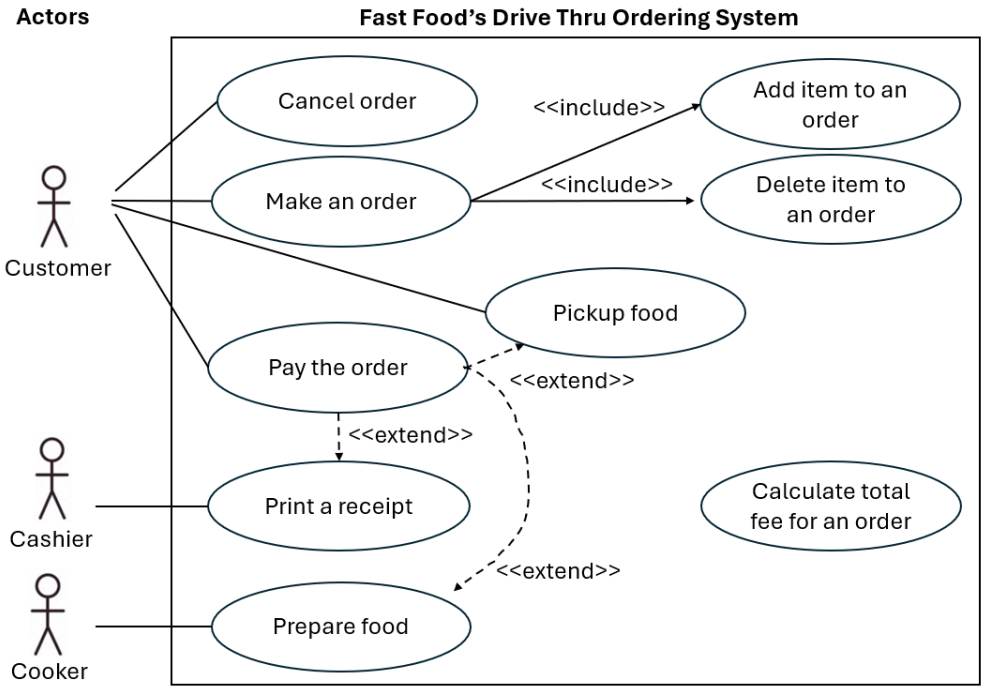


Figure 2.1 Use Case Diagram for the Fast-Food Drive-Thru Ordering System

2.4.1 Individual Use-Case descriptions

2.4.1.1 Use Case: make an order

Actors: customer

Preconditions: A machine displays McDonald's menu

Description: A machine starts in the waiting state. A customer selects items. The machine displays the total fee for the order.

Exceptions:

Deleted: if the customer selects an item and press "delete" button, the item will be removed from the order and the fee will be updated.

Cancelled: if the customer presses the cancel button, the order will be cleared.

Post conditions: the machine is ready for a new customer

2.4.1.2 Use Case: cancel an order

Actors: cashier

Preconditions: An order is waiting for a payment

Description: The order is displayed in a casher station computer. A customer cancels the order. The customer confirmed cancellation. The state of the order is “cancelled”. The fee is cleared

Exceptions:

Order does not exist: Cashier has admin power to manually remove the order.

An order cannot be cancelled if it was picked up.

Post conditions: the machine is ready for a new customer

2.4.1.3 Use Case: pay an order

Actors: customer

Preconditions: A cashier is waiting for a customer to pay the order

Description: The order is displayed in a casher station computer. A customer confirms the order and pays the order. The state of the order is “paid”.

Exceptions:

Cancelled: if the customer tells cashier that he/she likes to cancel the order, and the order will be cleared.

A customer cannot pay an order is already paid.

Post conditions: the machine is ready for a new customer

2.4.1.4 Use Case: print the receipt

Actors: cashier

Preconditions: A customer is waiting for a receipt

Description: The order is successfully paid. The machine prints out a receipt. Cashier hands the receipt to the customer

Exceptions:

Out of paper: cashier needs to place a new paper roll for printing receipt.

Out of ink: cashier needs to replace the ink

Post conditions: Customer is waiting for pick up food

2.4.1.5 Use Case: prepare food

Actors: McDonald's employees

Preconditions: A customer has paid the order

Description: The paid order is display on a machine near food preparing area. Employee starts to prepare the items listed on the order

Exceptions:

Out of material: employee needs to grab material from the storage room.

Food machine is broken: employee asks the customer for a replacement.

Post conditions: Food is ready for pick up

2.4.1.6 Use Case: pick up the order

Actors: customer

Preconditions: A customer is waiting for pick up the food

Description: The order is ready for pick up. Employee hands the food to the customer.

Exceptions:

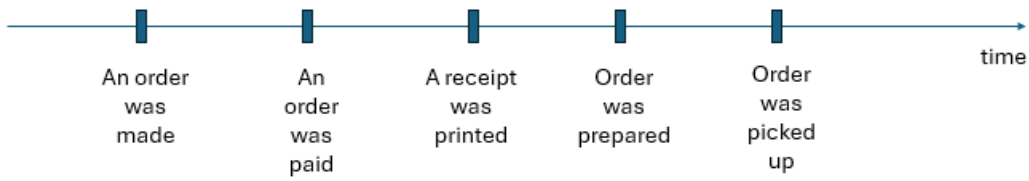
Food dropped: employee asks customer to wait at pick up zone while preparing another one.

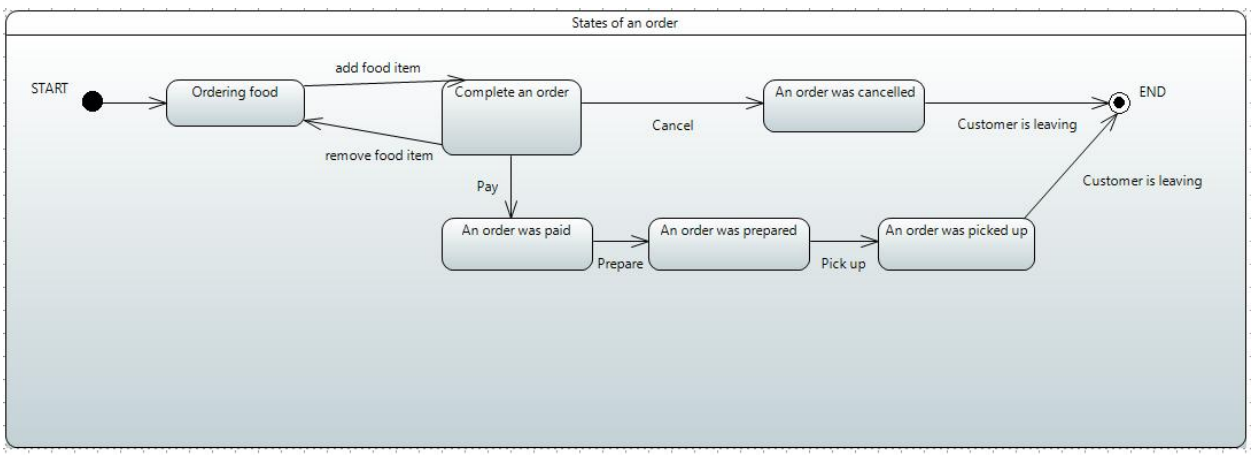
Forget about sauce: employee hands ketchup.

Post conditions: Customer leaves

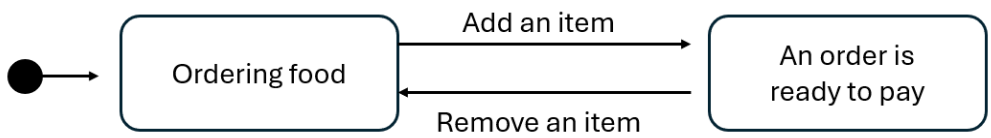
2.4.2 Use-Case State Model

A happy path of a successful ordering include: make an order, pay an order, print a receipt, prepare food, and order was picked up. The states of an order cycles from ordering, paid, preparing, ready to pick up, and picked up. An order can also be cancelled.





2.4.2.1 State Diagram for Use Case: make an order



- State Name: Ordering

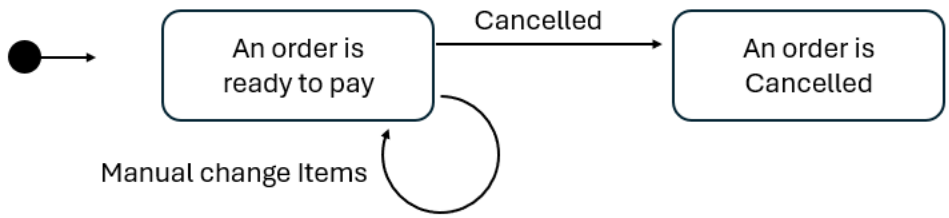
  State Description: a customer is ordering foods
  Event Sequence:
  - Add an item
  - Remove an item
  - Calculate total fee

  Condition that characterizes this state: the length of the order is changing
  Events accepted in this state:
  - Add an item
  - Remove an item

- State Name: Ready to pay

  State Description: a customer is happy with the order and ready to pay
  Event Sequence:
  Add an item
  Remove an item
  Calculate total fee
  Condition that characterizes this state: the length of the order is fixed
  Events accepted in this state:
  Add an item
  Remove an item

2.4.2.2 State Diagram for Use Case: cancel an order



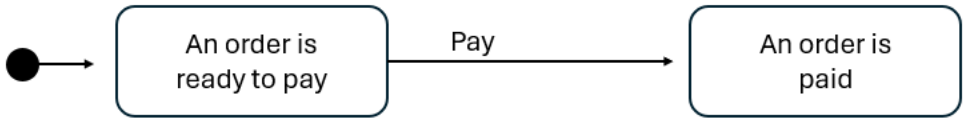
- State Name: Ready to pay

  State Description: an order is ready to pay
  Event Sequence:
  Cancel the entire order
  Calculate total fee
  Condition that characterizes this state: the length of the order is decreasing
  Events accepted in this state:
  Cancel the entire order
  A cashier can help to manually add / remove items

- State Name: Cancelled

  State Description: an order is cancelled
  Event Sequence:
  Clear the order
  Condition that characterizes this state: the length of the order is 0
  Events accepted in this state:
  Cancel the entire order

2.4.2.3 State Diagram for Use Case: pay an order



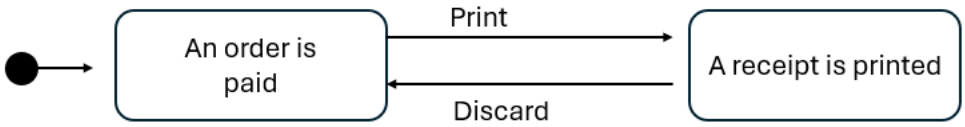
- State Name: Ready to pay

  State Description: an order is ready to pay
  Event Sequence:
  Calculate total fee
  Pay an order
  Condition that characterizes this state: the length of the order is fixed
  Events accepted in this state:
  Calculate total fee
  Pay an order

- State Name: Paid

  State Description: an order is paid
  Event Sequence:
  Clear the total fee of an order
  Add to restaurant food preparing queue
  Condition that characterizes this state: the cost of the order is 0
  Events accepted in this state:
  Clear the total fee of an order

2.4.2.4 State Diagram for Use Case: print the receipt



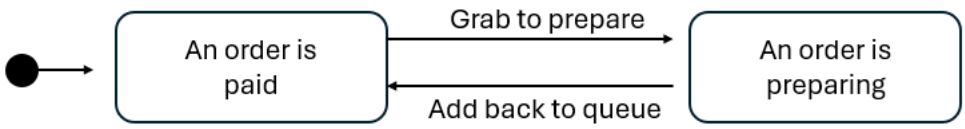
- State Name: Paid

  State Description: an order is paid
  Event Sequence:
  Clear the total fee of an order
  Add to restaurant food preparing queue
  Condition that characterizes this state: the cost of the order is 0
  Events accepted in this state:
  Clear the total fee of an order
  Print a receipt

- State Name: Printed

  State Description: a receipt is printed
  Event Sequence:
  Calculate total fee
  Print a receipt
  Condition that characterizes this state: the order is fixed
  Events accepted in this state:
  Calculate total fee
  Print a receipt
  Discard a receipt

2.4.2.5 State Diagram for Use Case: print the receipt



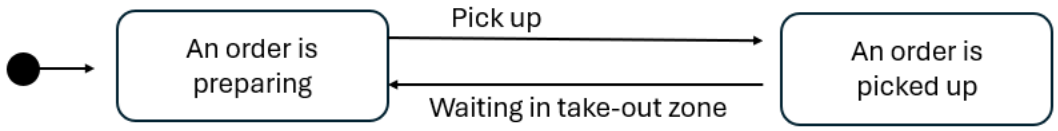
- State Name: Paid

  State Description: an order is paid
  Event Sequence:
  Clear the total fee of an order
  Add to restaurant waiting queue
  Condition that characterizes this state: the cost of the order is 0
  Events accepted in this state:
  Clear the total fee of an order
  Grab to prepare
  Add back to waiting queue

- State Name: Preparing

  State Description: an order is preparing
  Event Sequence:
  Add to waiting queue
  Grab to prepare
  Condition that characterizes this state: the order is processing
  Events accepted in this state:
  Add to waiting queue
  Grab to prepare

2.4.2.6 State Diagram for Use Case: pick up the order



- State Name: Preparing

  State Description: an order is preparing
  Event Sequence:
  - Add to waiting queue
  - Grab to prepare

  Condition that characterizes this state: the order is processing
  Events accepted in this state:
  - Add to waiting queue
  - Grab to prepare

- State Name: Picked up

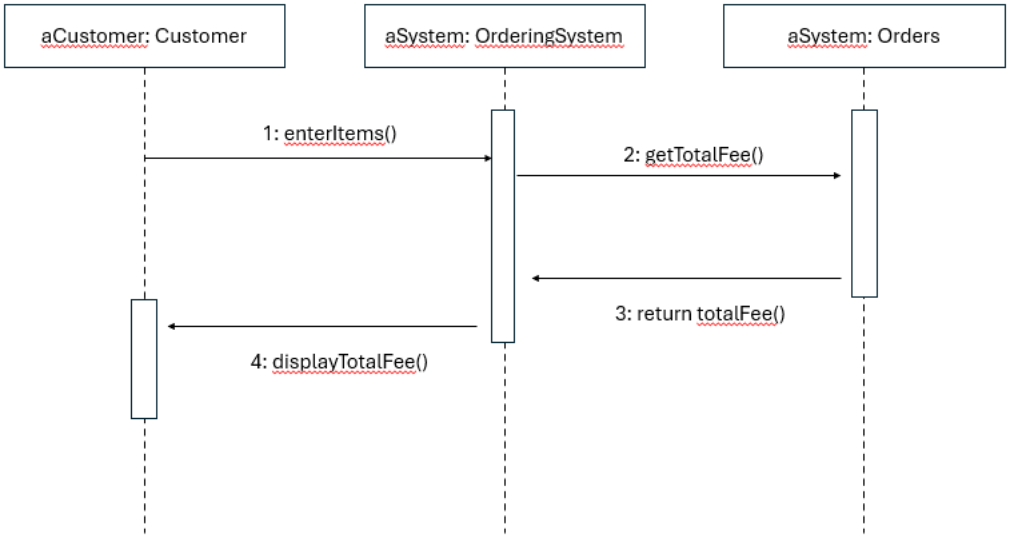
  State Description: a customer picks up the order
  Event Sequence:
  - Mark the order is completed
  - Remove the order from the food preparing queue

  Condition that characterizes this state: the order is closed
  Events accepted in this state:
  - Pick up the order
  - Continue to wait in the take-out zone

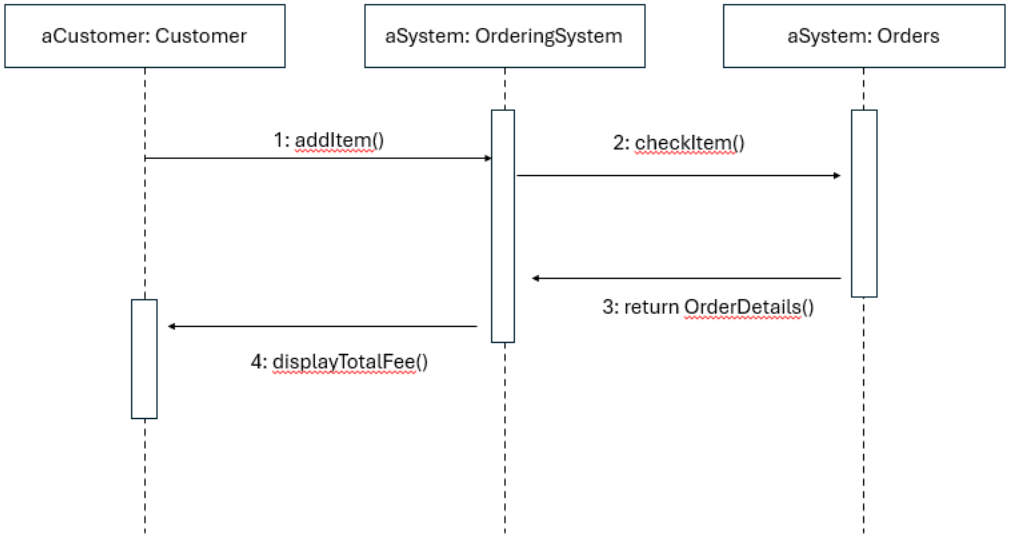
2.4.3 Interaction Model (Sequence Diagrams)

The sequence-diagrams for the above use cases are listed below.

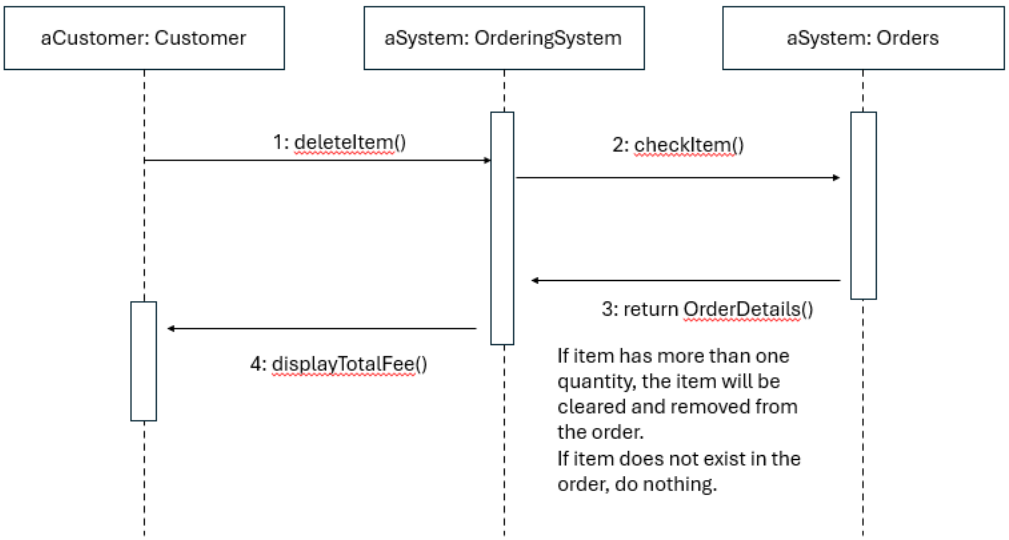
2.4.3.1 Sequence diagram: make an order



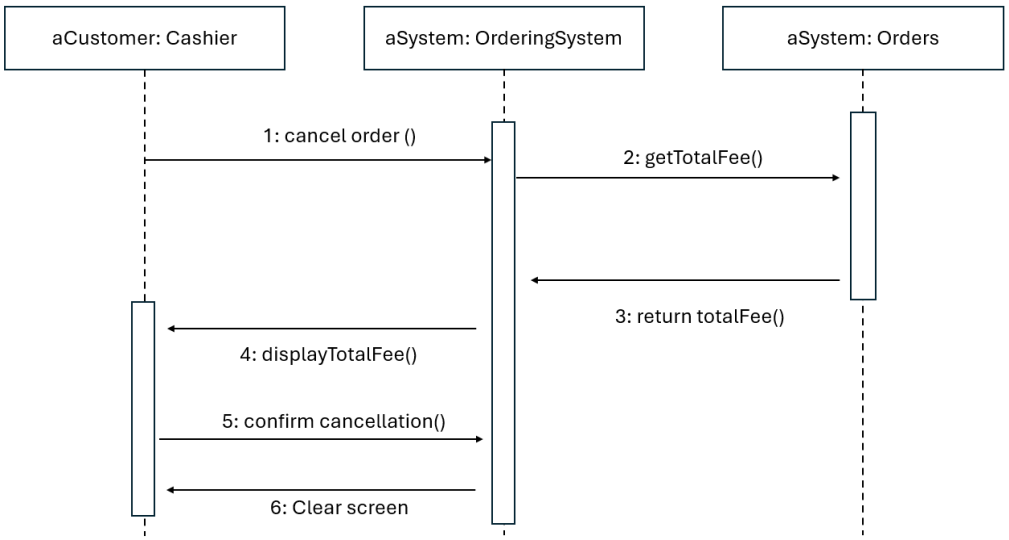
Sequence diagram: add an item to the order



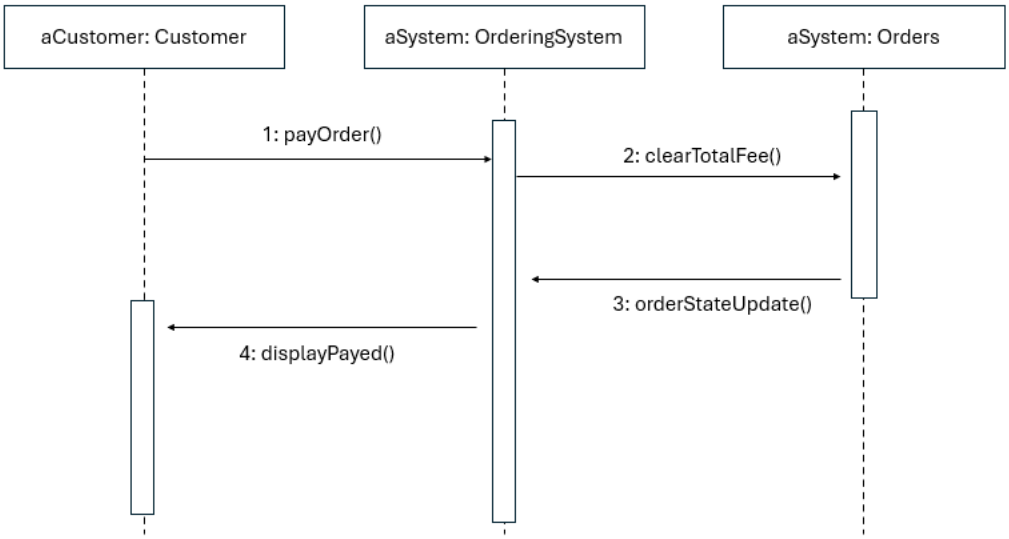
Sequence diagram: delete an item to the order



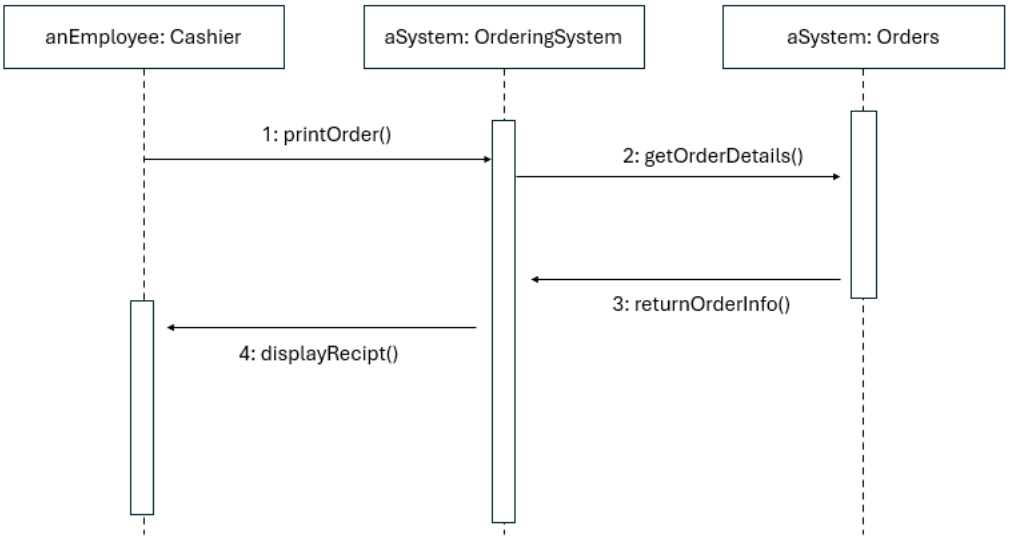
2.4.3.2 Sequence diagram: cancel an order



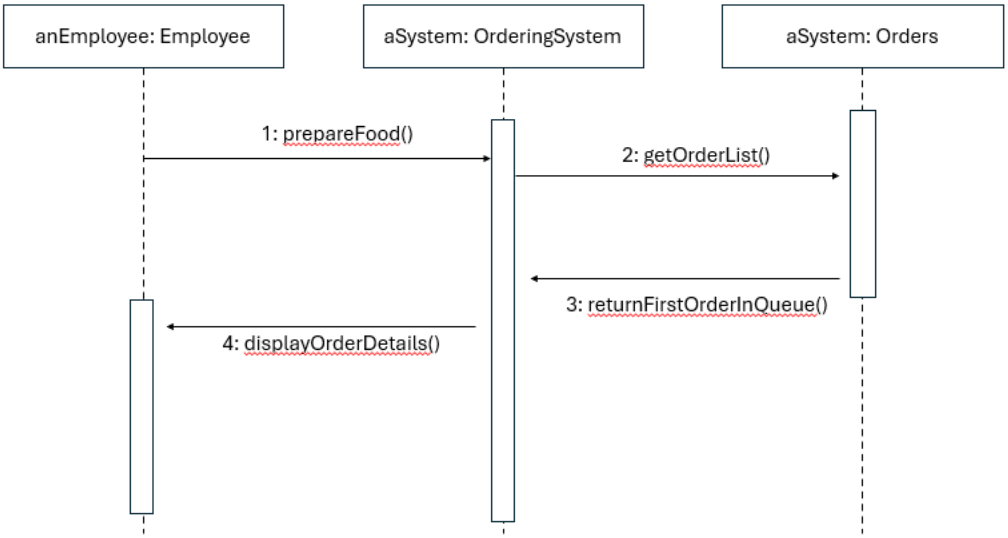
2.4.3.3 Sequence diagram: pay an order



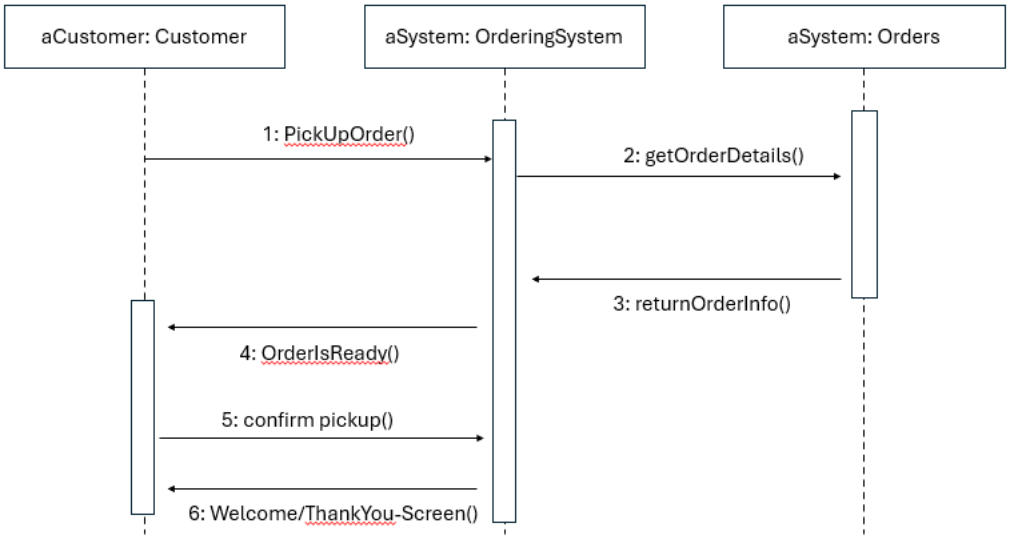
2.4.3.4 Sequence diagram: print a receipt



2.4.3.5 Sequence diagram: prepare food

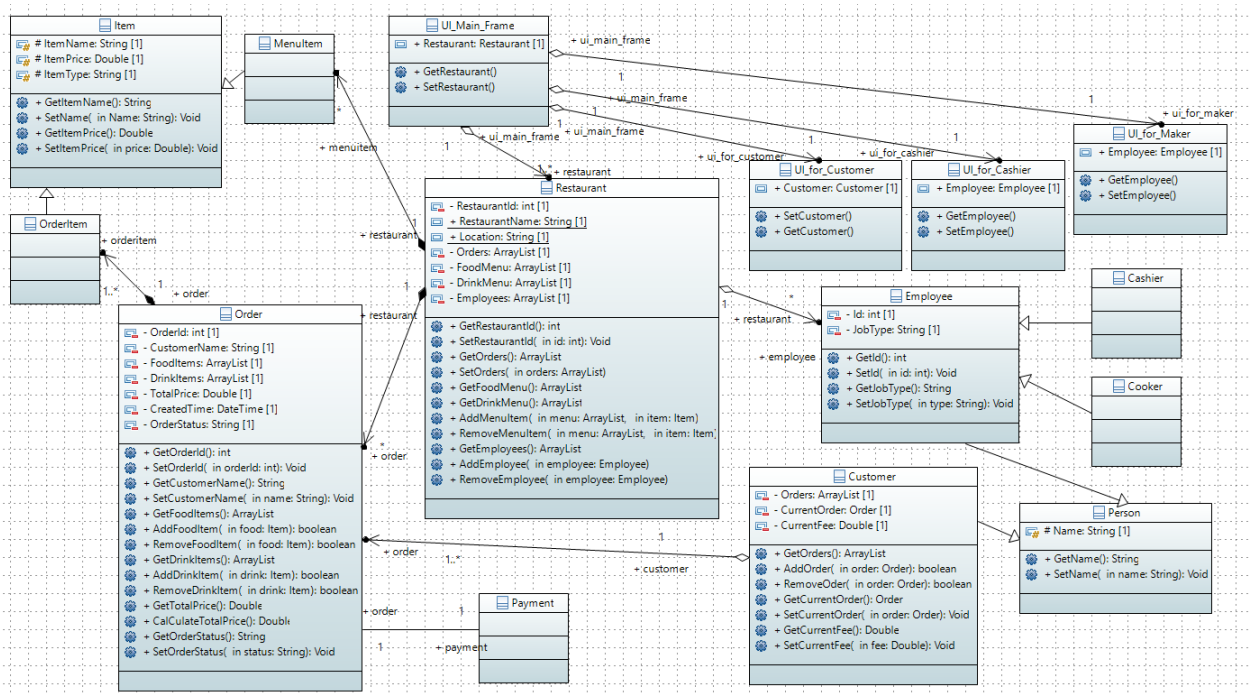


2.4.3.6 Sequence diagram: pick up the order

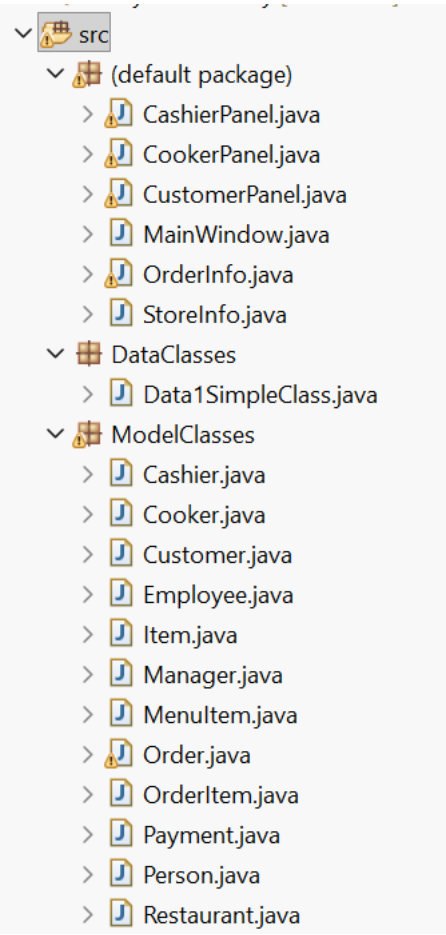


2.5 Domain Class Model

The domain class model for a Fast-Food drive-thru system includes following two categories of classes: UI related classes and data model related classes.



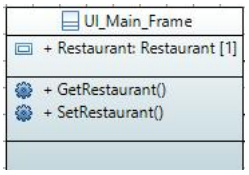
Therefore, the domain class model contains three packages: default package (including UI classes), ModelClasses package (including model classes), and DataClasses package (including testing data).



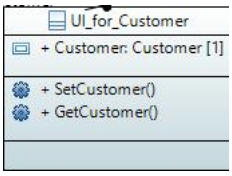
The details of the classes are illustrated below.

2.5.1 Default Package (including UI Classes)

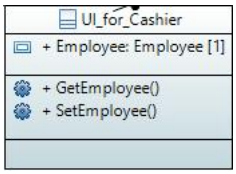
- MainWindow.java



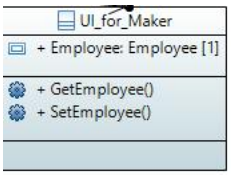
- CustomerPanel.java



- CashierPanel.java

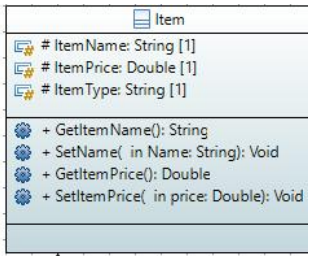


- CookerPanel.java

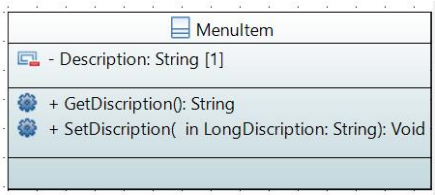


2.5.2 Model Classes

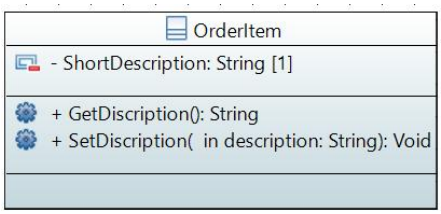
- Item.java



- MenuItem.java



- OrderItem.java



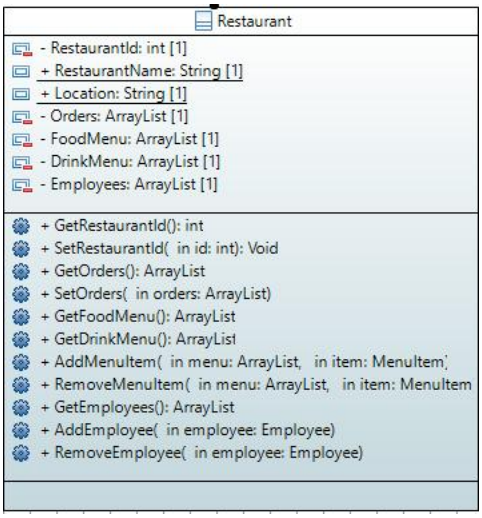
- Payment.java

```
Payment
- PaymentId: int [1]
- PayMethod: String [1]
- PayAmount: Double [1]
- PayStatus: String [1]
- PayTime: DateTime [1]
- AuthCode: String [1]

+ GetPaymentId(): int
+ SetPaymentId( in id: int): Void
+ GetPayMethod(): String
+ SetPayMehtod( in method: String)
+ GetPayAmount(): Double
+ SetPayAmount( in amount: Double): Void
+ GetStatus(): String
+ SetStatus( in status: String)
+ GetPayTime(): DateTime
+ SetPayTime( in time: DateTime): Void
+ GetAuthCode(): String
+ SetAuthCode( in code: String): Void
+ Refund(): boolean
+ Pay(): boolean
+ Authorize(): boolean
+ Capture(): boolean
```

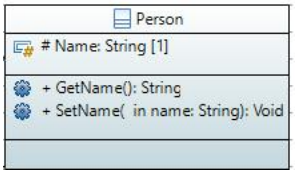
- Order.java

```
Order
- OrderId: int [1]
- CustomerName: String [1]
- FoodItems: ArrayList [1]
- DrinkItems: ArrayList [1]
- TotalPrice: Double [1]
- CreatedTime: DateTime [1]
- OrderStatus: String [1]

+ GetOrderId(): int
+ SetOrderId( in orderId: int): Void
+ GetCustomerName(): String
+ SetCustomerName( in name: String): Void
+ GetFoodItems(): ArrayList
+ AddFoodItem( in food: MenuItem): boolean
+ RemoveFoodItem( in food: MenuItem): boolean
+ GetDrinkItems(): ArrayList
+ AddDrinkItem( in drink: MenuItem): boolean
+ RemoveDrinkItem( in drink: MenuItem): boolean
+ GetTotalPrice(): Double
+ CalCulateTotalPrice(): Doubl
+ GetOrderStatus(): String
+ SetOrderStatus( in status: String): Void
```

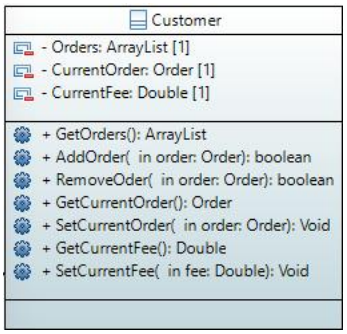
- Restaurant.java



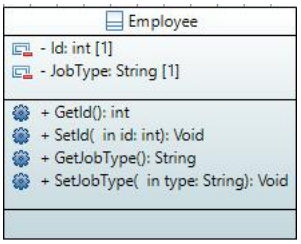
- Person.java



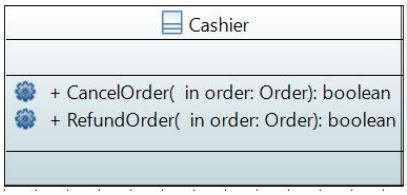
- Customer.java



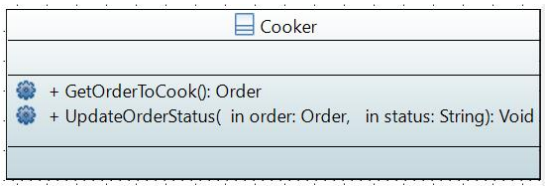
- Employee.java



- Cashier.java



- Cooker.java



2.6 Testing

2.6.1 Unit Testing

Added unit tests for module classes. For example: testing get/set and other methods.

UnitTests.java | ModuleTests.java | Data1SimpleClass.java | Order.java

```java
package DataClasses;

import java.util.*;
import ModelClasses.*;

public class UnitTests {

    public static void main(String[] args)
    {
        System.out.println("Unit Testing ...");
        System.out.println("");

        // Testing Order.java
        Order order1 = new Order();
        order1.SetCustomer("customer1");
        MenuItem MenuItem11 = new MenuItem("Hamburger", "Standard Burger", 3.00, "food");
        MenuItem MenuItem21 = new MenuItem("Tea", "Iced Tea", 1.50, "drink");
        order1.AddToAllItems(MenuItem11);
        order1.AddToAllItems(MenuItem21);
        System.out.println("Testing GetOrderAllItemsInfo()");
        System.out.println(order1.GetOrderAllItemsInfo());
        System.out.println("Testing GetAllItemPrice()");
        System.out.println("Total Price = $" + order1.GetAllItemPrice());
    }
}
```

```
Unit Testing ...

Testing GetOrderAllItemsInfo()
Hamburger ($3.0)
Tea ($1.5)

Testing GetAllItemPrice()
Total Price = $4.5
```

2.6.2 Module Testing

Added unit tests for module classes. For example: testing an array list of orders.

UnitTests.java | ModuleTests.java | Data1SimpleClass.java | Order.java | MainWindow.java

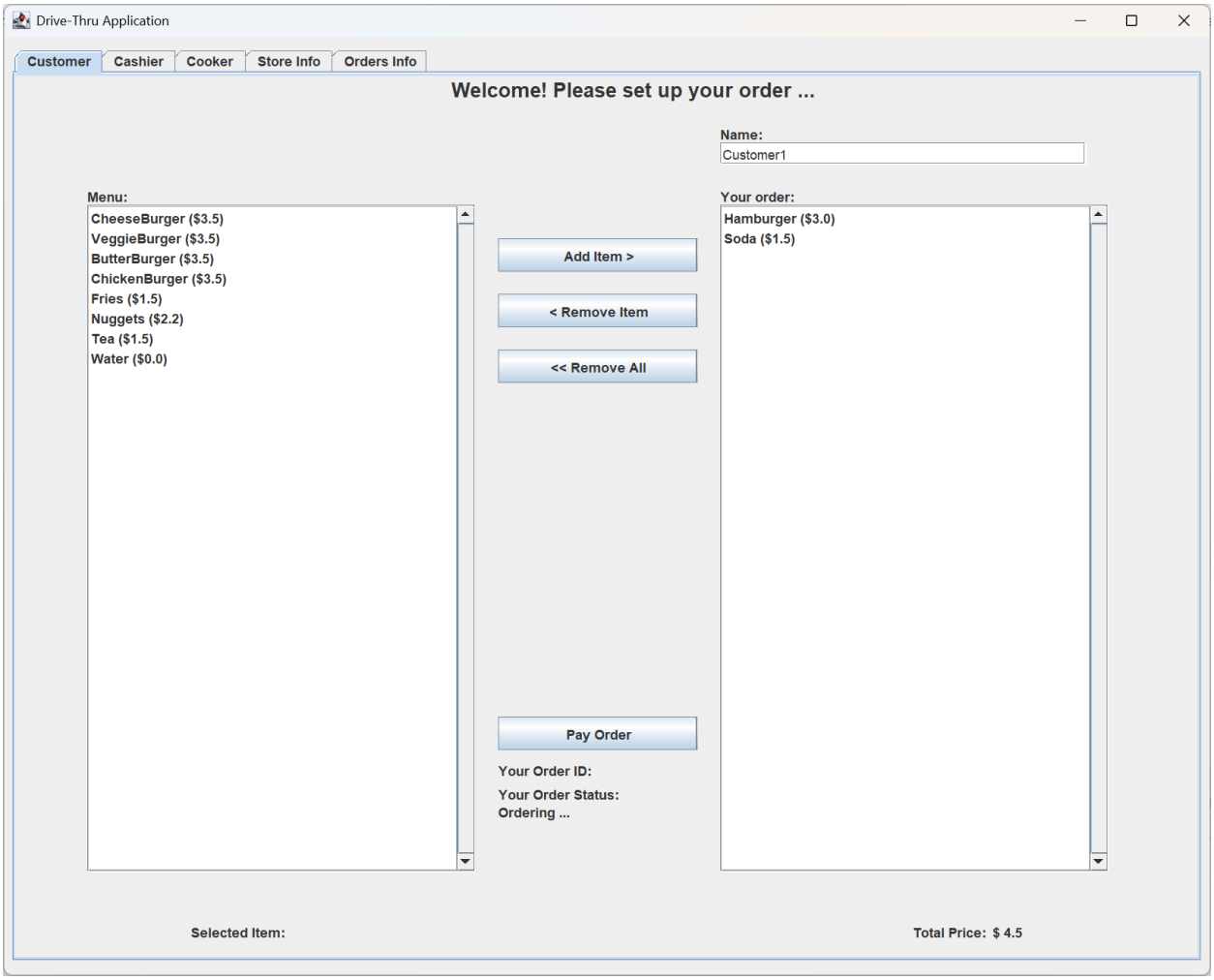
```java
public class ModuleTests {

    public static void main(String[] args)
    {
        System.out.println("Module Testing ...");
        System.out.println("");

        // Testing order list functions
        Order order1 = new Order();
        order1.SetOrderId(1);
        order1.SetCustomer("customer1");
        MenuItem MenuItem11 = new MenuItem("Hamburger", "Standard Burger", 3.00, "food");
        MenuItem MenuItem21 = new MenuItem("Tea", "Iced Tea", 1.50, "drink");
        order1.AddToAllItems(MenuItem11);
        order1.AddToAllItems(MenuItem21);
        order1.SetOrderStatus("Paid");

        Order order2 = new Order();
        order2.SetOrderId(2);
        order2.SetCustomer("customer2");
        MenuItem MenuItem12 = new MenuItem("Soda", "Soda", 1.50, "drink");
        MenuItem MenuItem22 = new MenuItem("Fries", "Fries", 1.50, "food");
        order2.AddToAllItems(MenuItem12);
        order2.AddToAllItems(MenuItem22);
        order2.SetOrderStatus("Preparing");

        ArrayList<Order> RestaurantOrders = new ArrayList<Order>();

        System.out.println("Added Order 1:");
        RestaurantOrders.add(order1);
        for(Order i: RestaurantOrders) {
            System.out.println("Order ID=" + i.GetOrderId() + ", Status=" + i.GetOrderStatus());
        }

        System.out.println("");
        System.out.println("Added Order 2:");
        RestaurantOrders.add(order2);
        for(Order i: RestaurantOrders) {
            System.out.println("Order ID=" + i.GetOrderId() + ", Status=" + i.GetOrderStatus());
        }

    }
}
```

```
Module Testing ...

Added Order 1:
Order ID=1, Status=Paid

Added Order 2:
Order ID=1, Status=Paid
Order ID=2, Status=Preparing
```

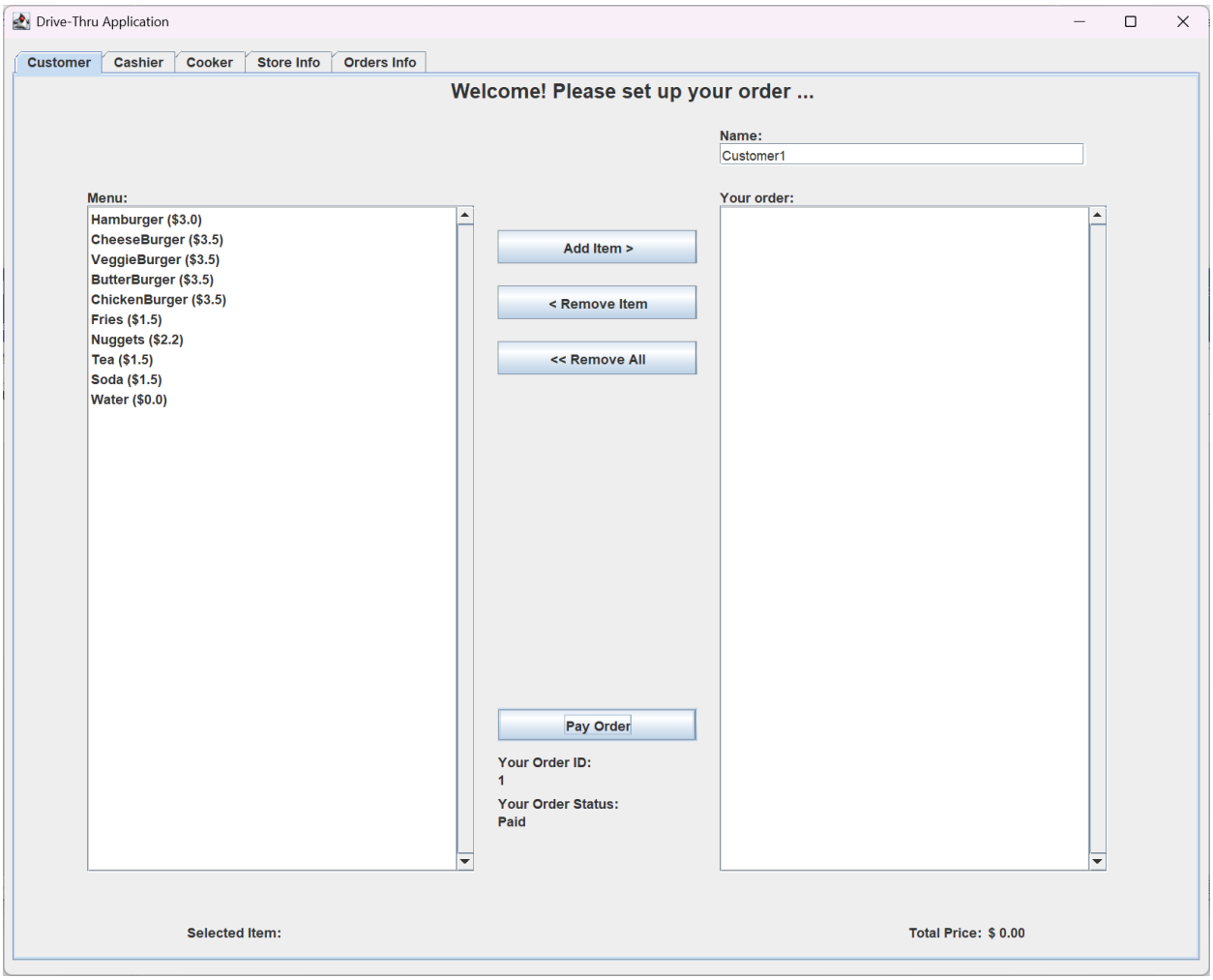
2.6.3 Use-Case Testing

Use UI performed use-case tests.

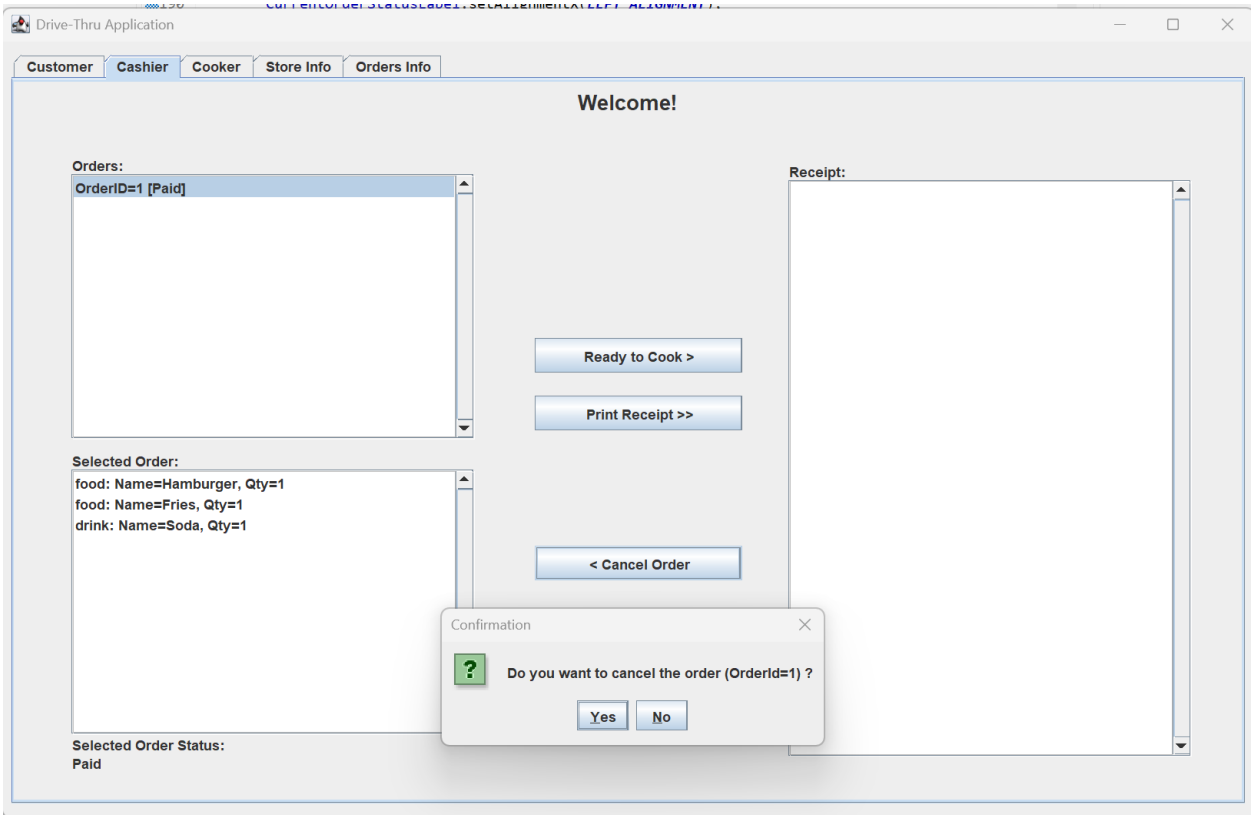
For example, use-case: make an order by adding or removing items

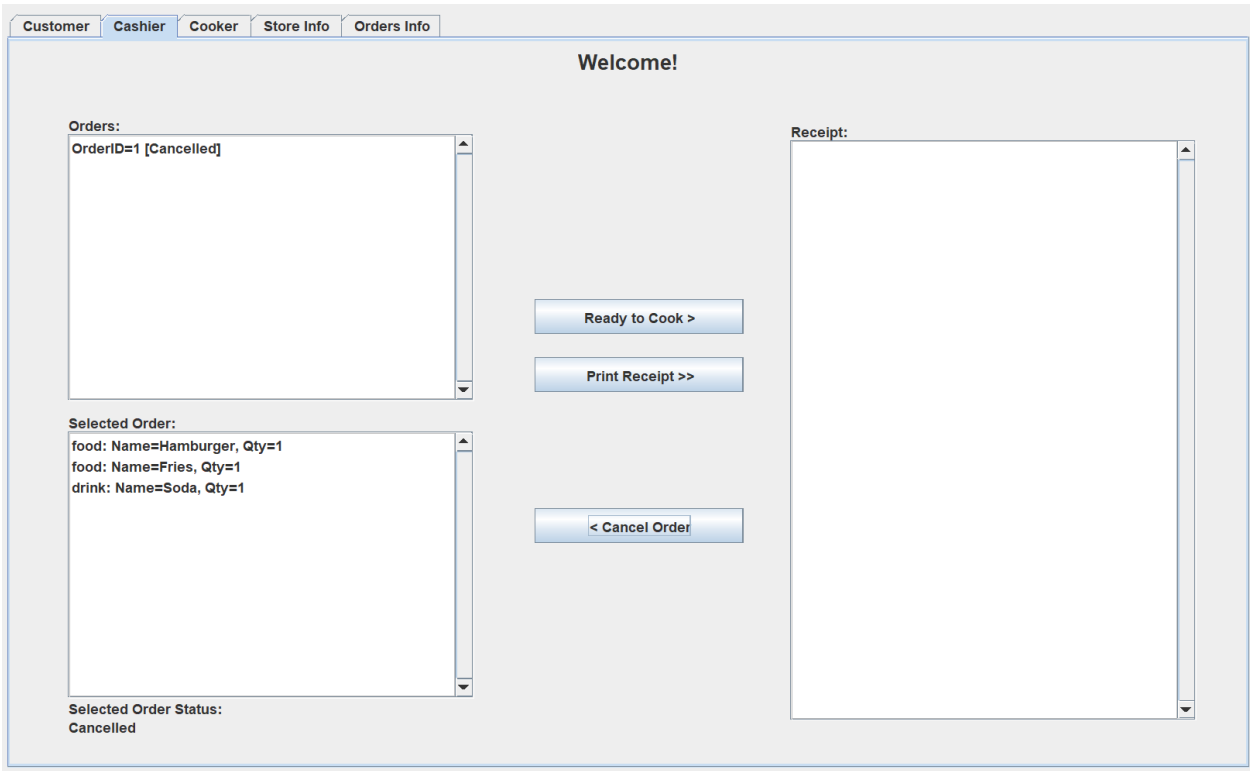


For example, use case: pay an order

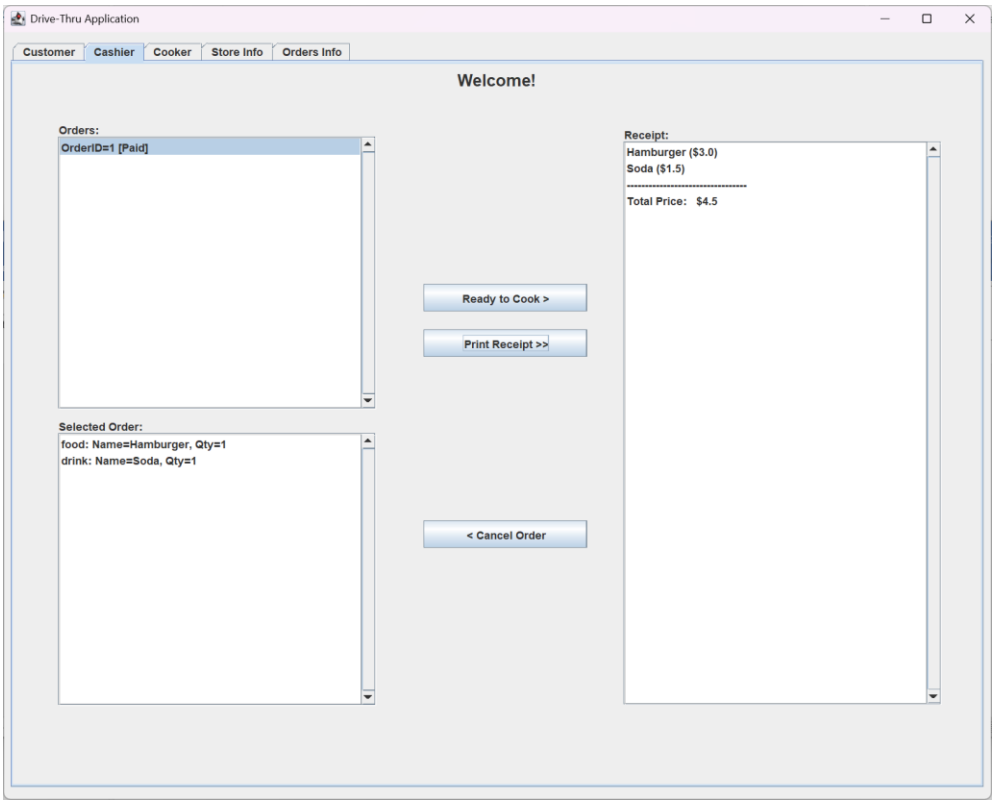


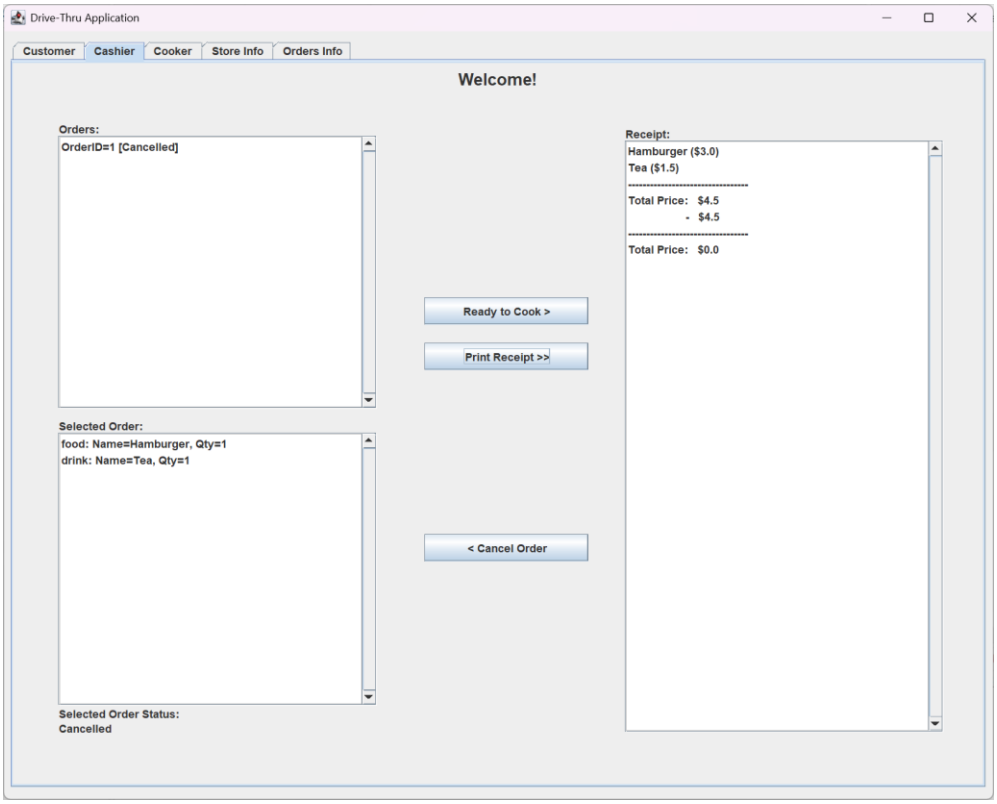
For example, use case: cancel an order



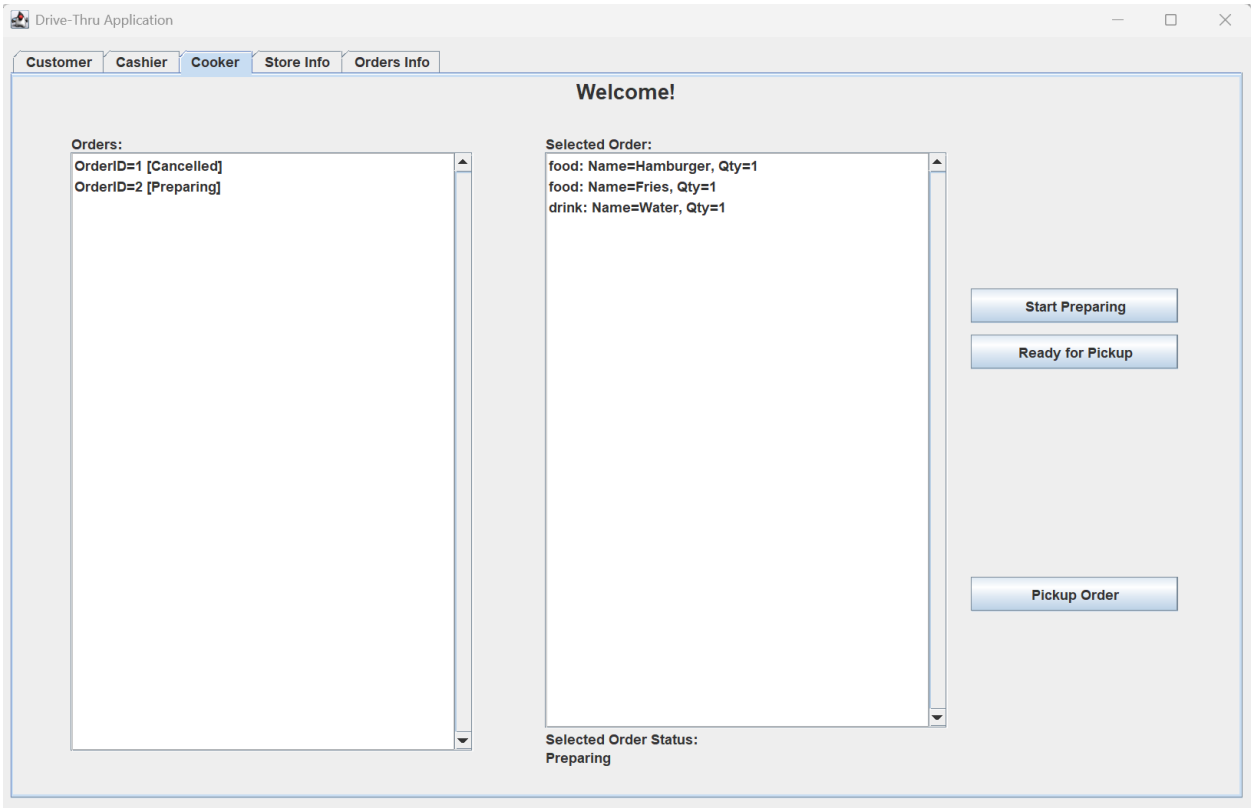


For example, use case: print a receipt

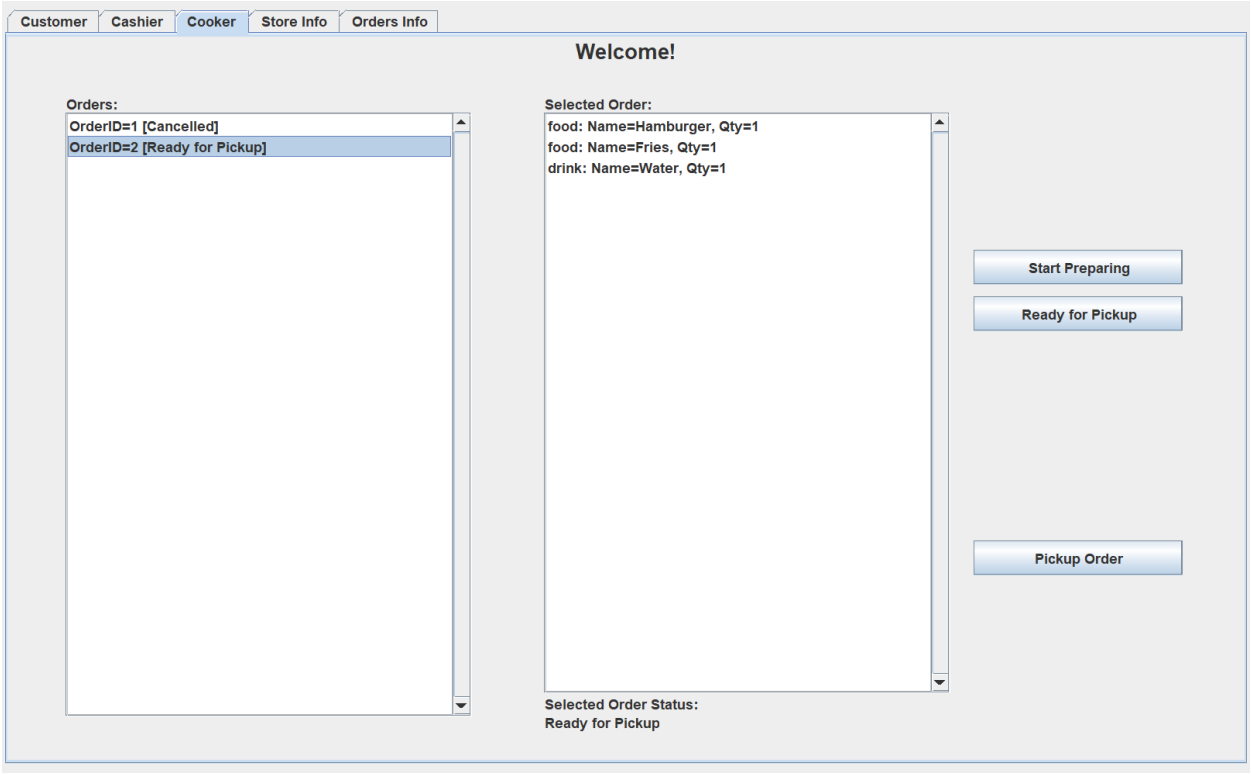


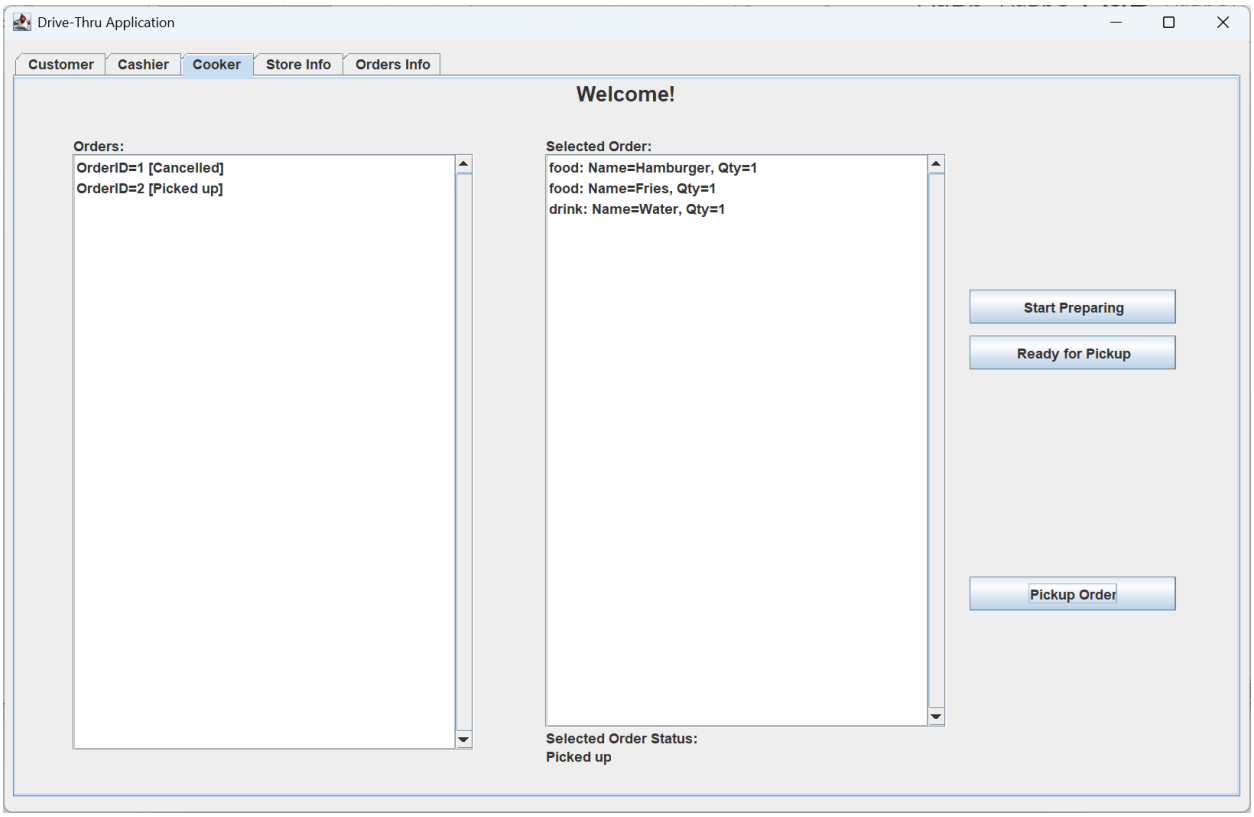


For example, use case: prepare food



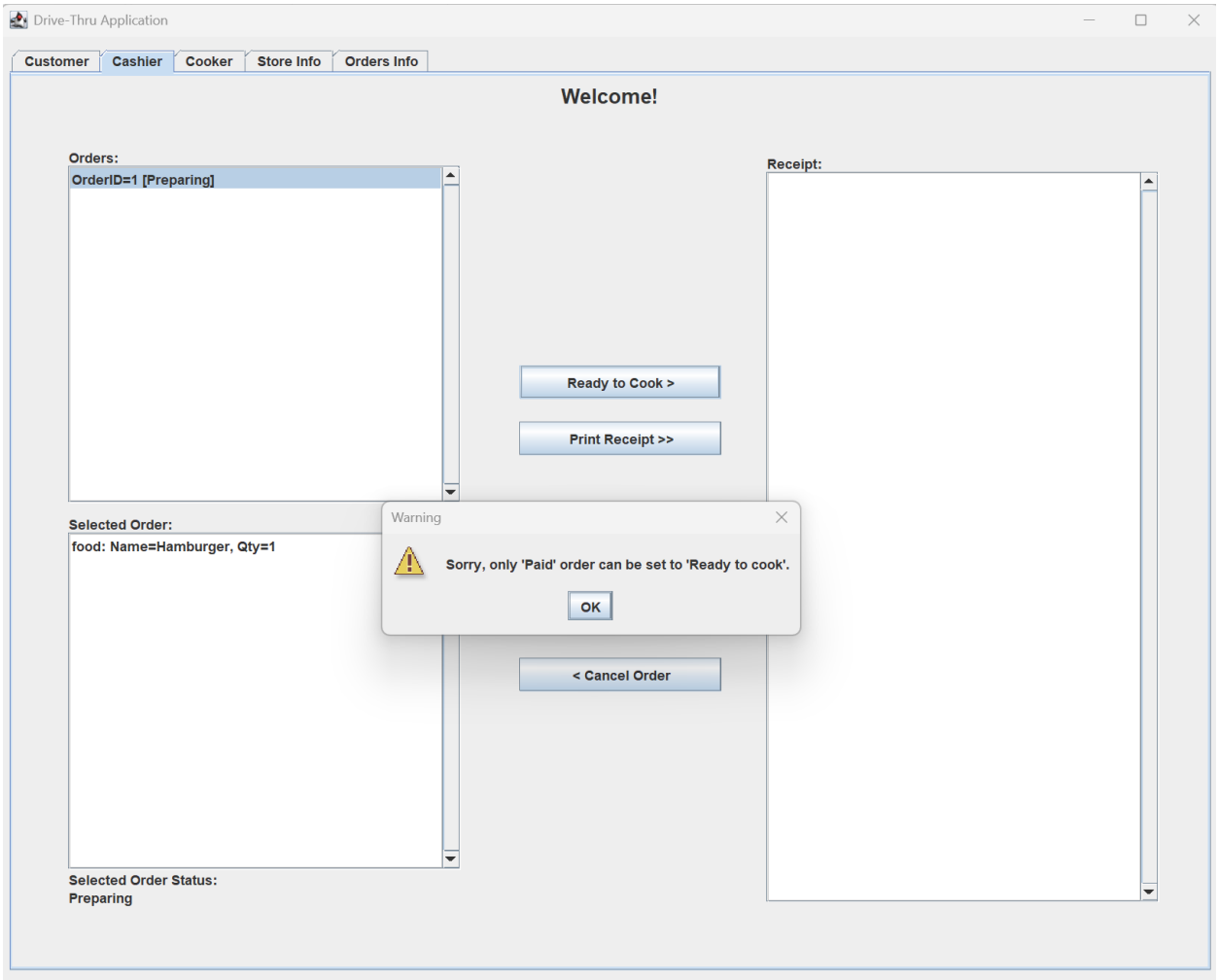
For example, use case: pick up the order





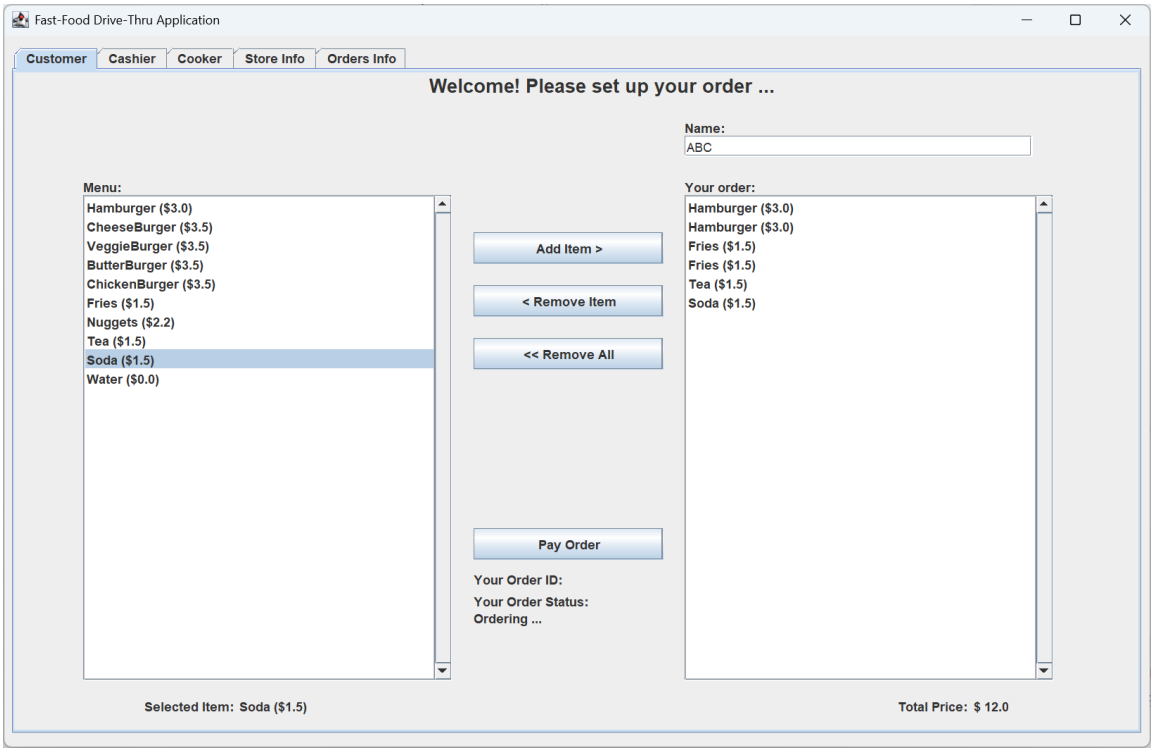
Also used UI for exception testing.

For example: only “Paid” orders can move to “Ready to Cook” state.

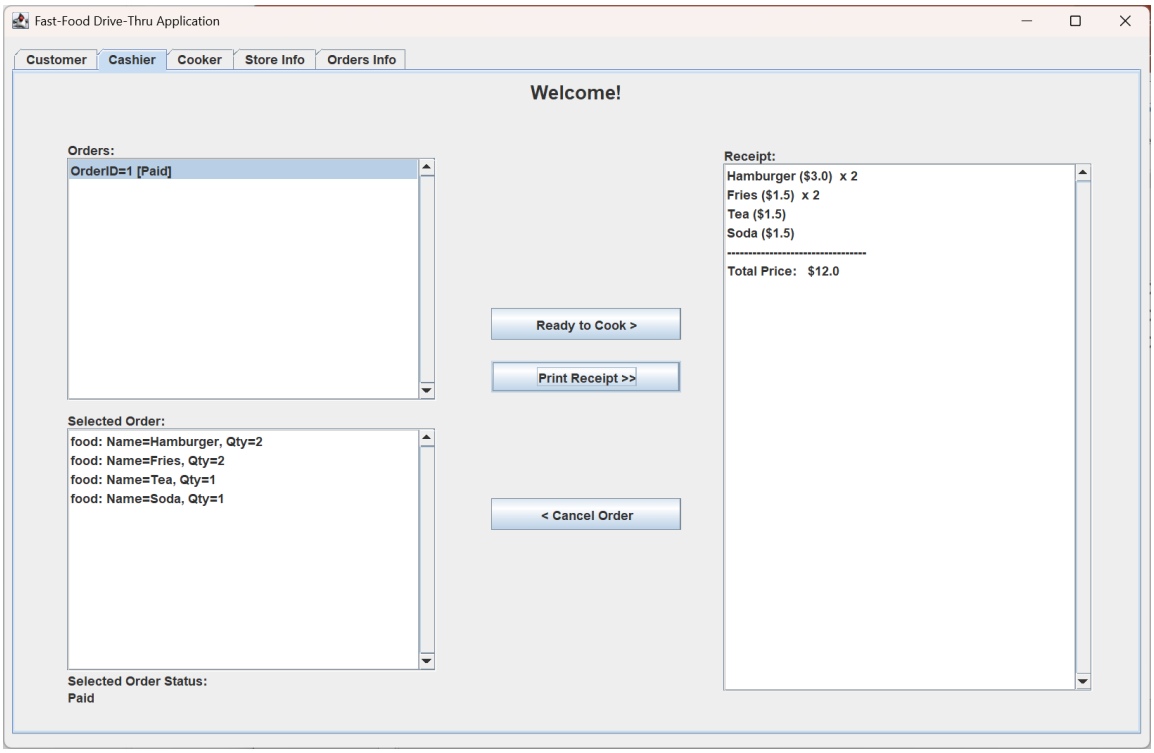


2.7 User Interfaces (UIs)

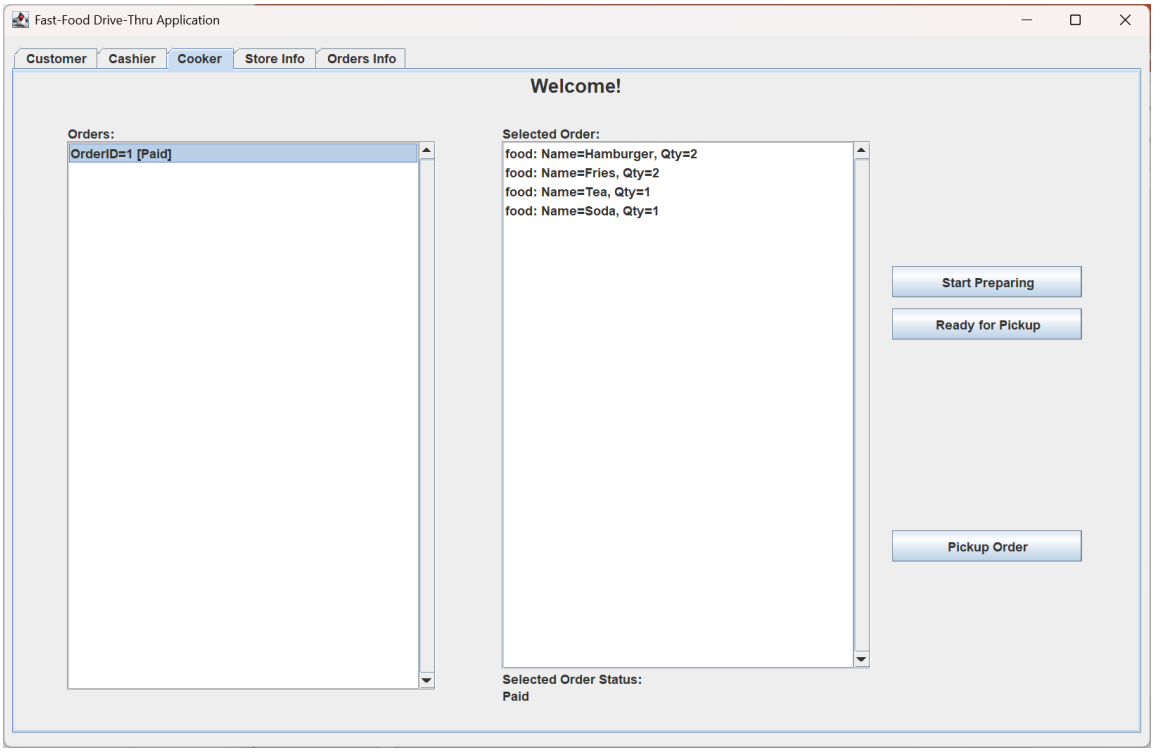
2.7.1 Customer UI



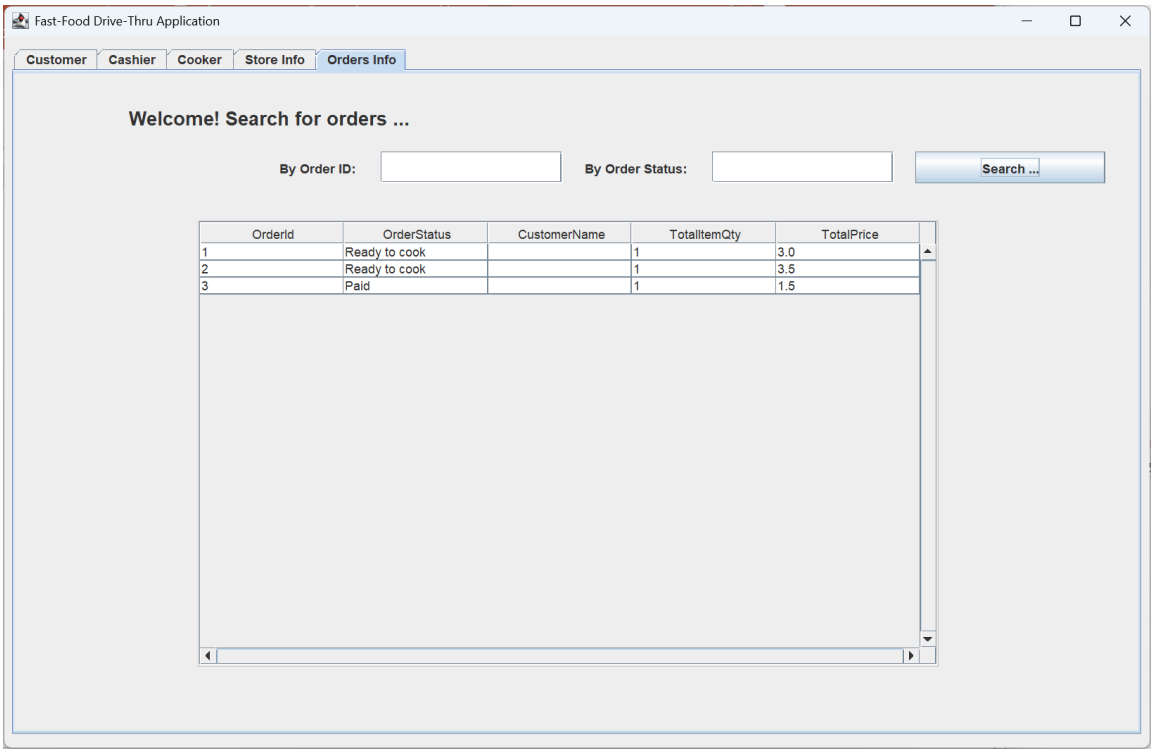
2.7.2 Cashier UI



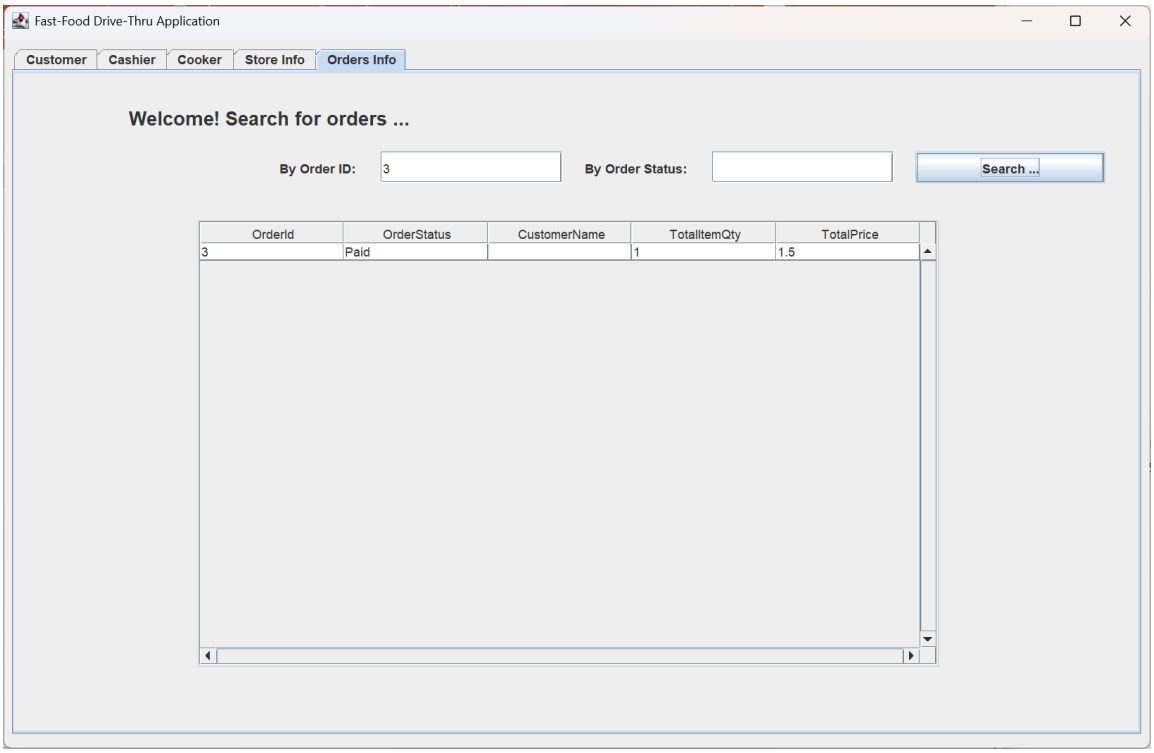
2.7.3 Cooker UI



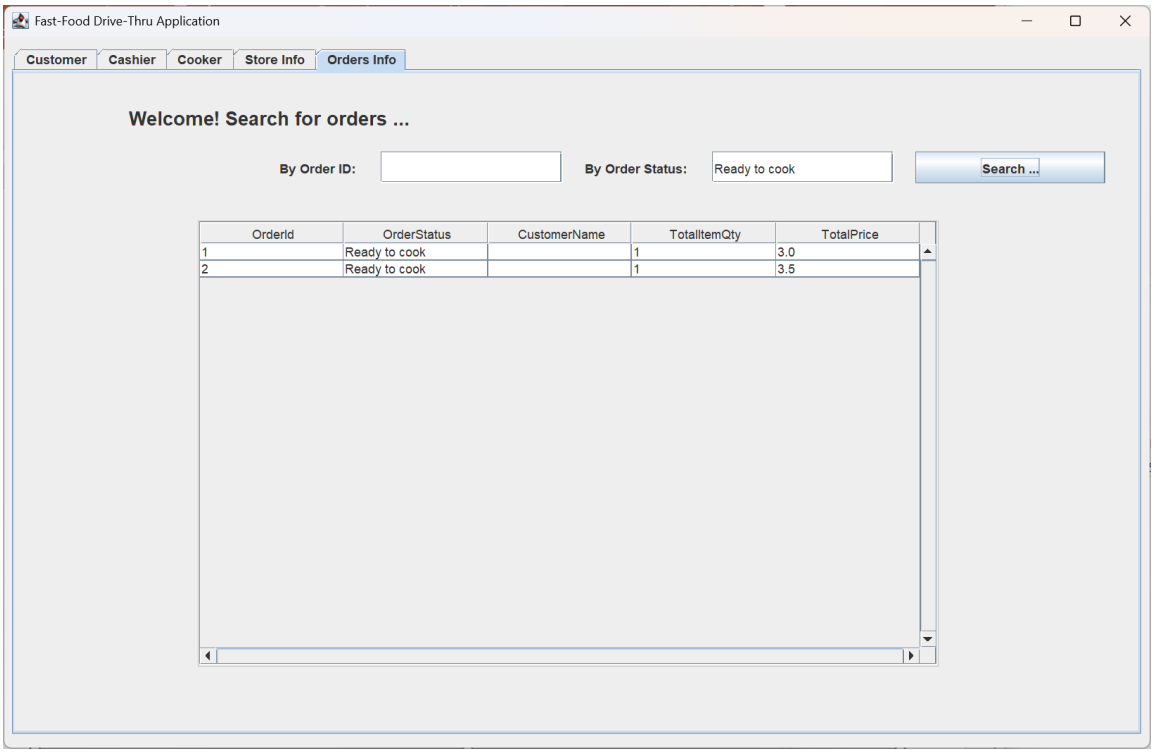
2.7.4 Orders Info UI (Bonus Feature)



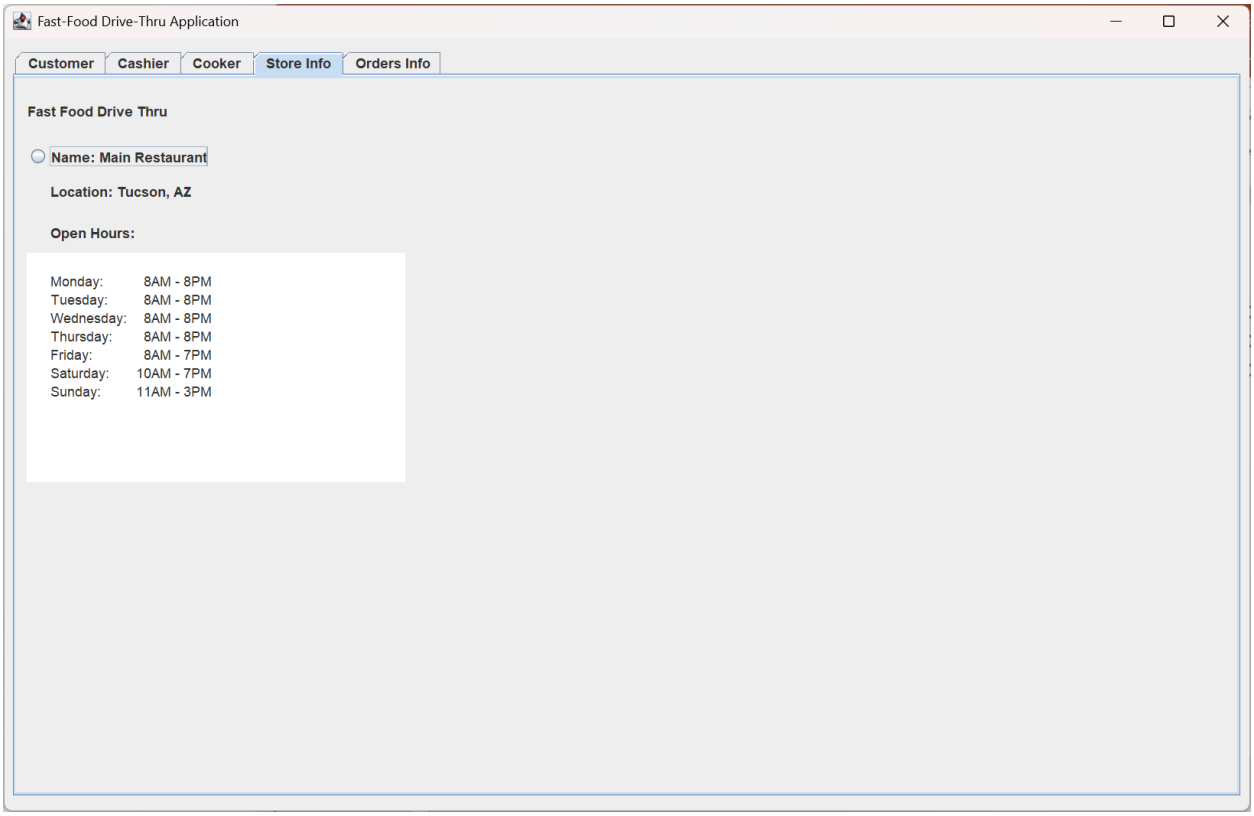
Search by Order ID



Search by Order Status



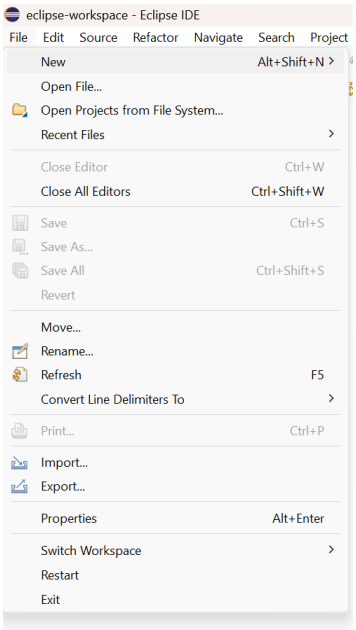
2.7.5 Store Info UI (Bonus Feature)



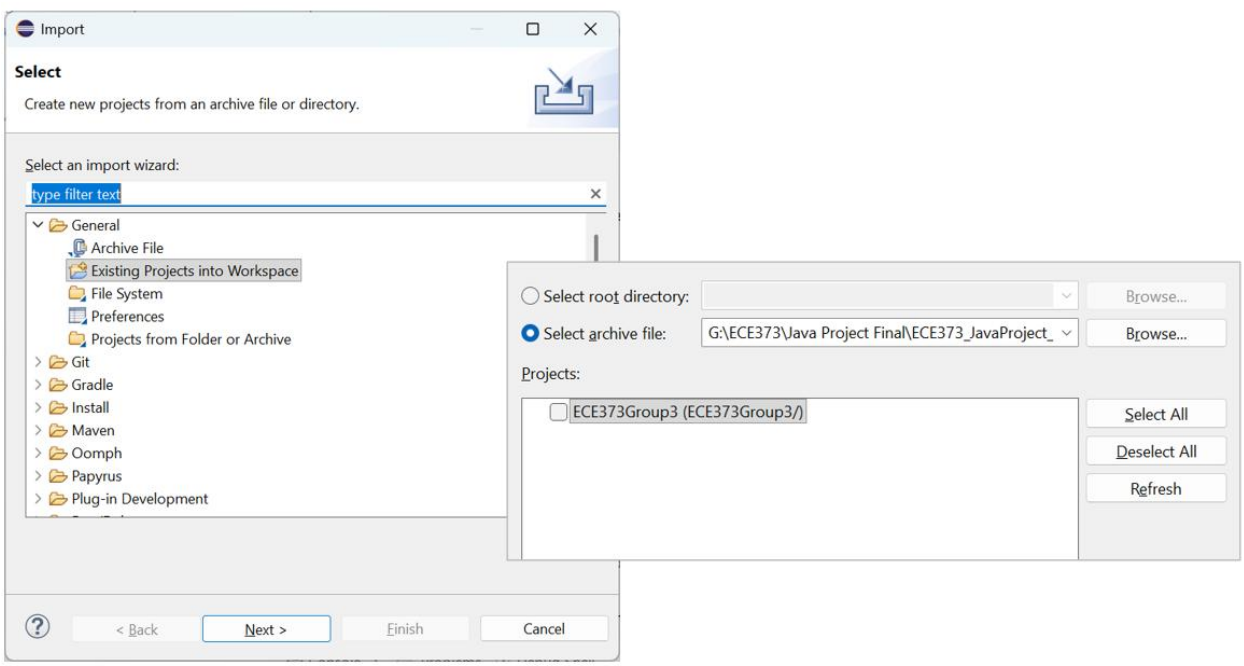
2.8 Software Install and Launch

Import the zip file to Eclipse IDE.

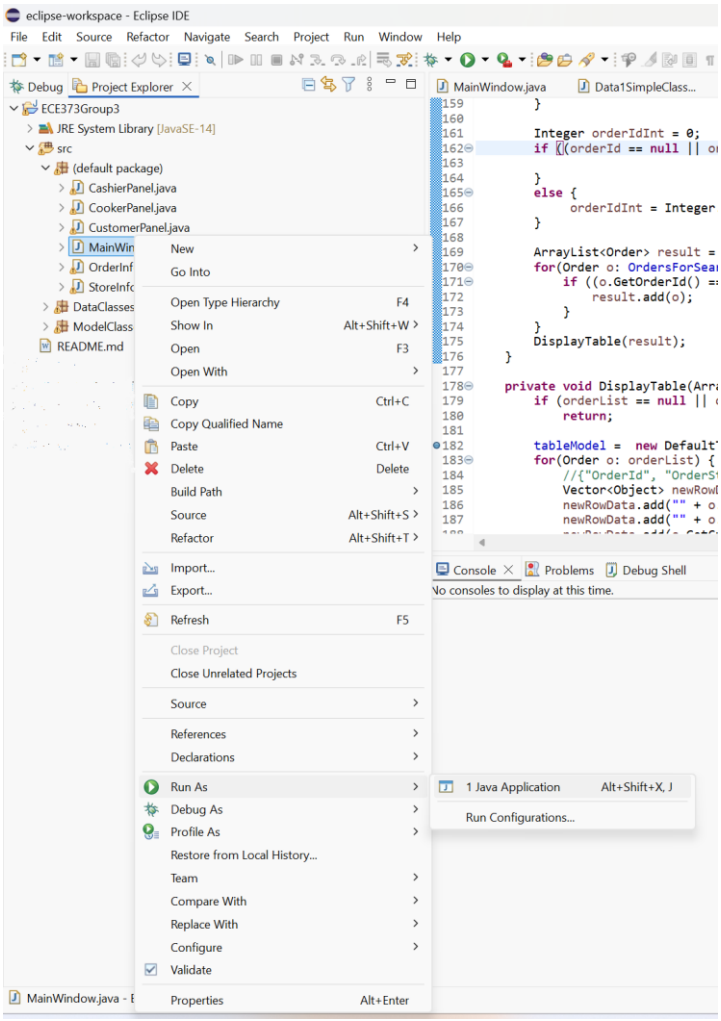
(a) Choose File → Import

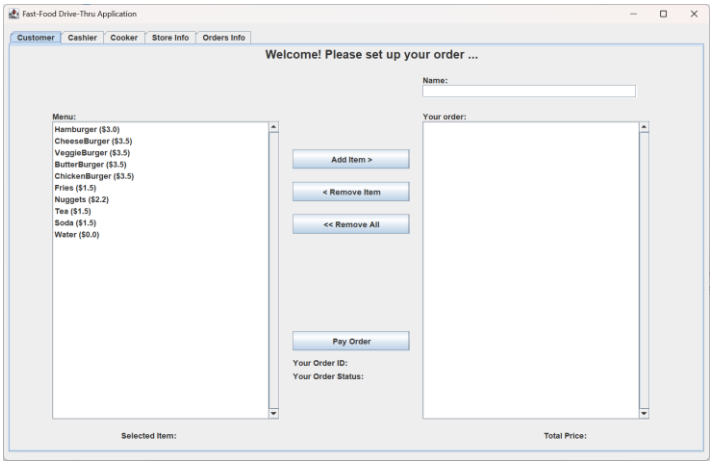


(b) Choose General → Existing Projects into Workspace
Browse to get the zip file



(c) Select “MainWindow.java”, then click “Run As” menu button from the menu bar to launch the desktop GUI app.





Deliverable 3: YouTube Video

YouTube Link:

https://youtu.be/SzET5pRDcLQ